University of Moratuwa

Department of Electronic and Telecommunication Engineering

EN3150

Assignment 03

Resource-Constrained CNN Design and Transfer Learning for DeFungi Image Classification

Group Members

Lavanathan.J	230371R
Panuharan.S	230462X
Peranavan.K	230474K
Rifnaz.K.R.M	230550P

Project repository: `https://github.com/rahumanrifnaz-dev/DeFungi`

Contents

1 Introduction **1**

2 Dataset and Data Preparation **1**
2.1 DeFungi Dataset . . . 1
2.2 Dataset Classes . . . 1
2.3 64×64 Input . . . 2
2.4 Stratified 70/15/15 Split . . . 2
2.5 Preprocessing and Augmentation . . . 2

3 Custom CNN Architectures **3**
3.1 Model A . . . 3
3.2 Model B . . . 3
3.3 Depthwise Separable Convolution . . . 4
3.4 Parameter Calculation . . . 4
3.5 Hardware-Aware Activation Selection . . . 4

4 Optimizer Selection and Tuning **4**
4.1 SGD . . . 4
4.2 SGD with Momentum . . . 4
4.3 Selected Adaptive Optimizer . . . 5
4.4 Experimental Comparison . . . 5
4.5 Selected Optimizer . . . 5

5 Custom CNN Training and Evaluation **5**
5.1 Training Setup . . . 5
5.2 Training Curves . . . 6
5.3 Test Results . . . 6
5.4 Confusion Matrices . . . 7
5.5 Model A vs Model B Resource Comparison . . . 7

6 Lightweight State-of-the-Art Models **8**
6.1 MobileNetV2 . . . 8
6.2 EfficientNet-B0 . . . 8
6.3 Transfer-Learning Strategy . . . 8
6.4 Results . . . 8

7 Final Comparison **10**
7.1 Accuracy . . . 10
7.2 Parameter Count . . . 10
7.3 Memory Footprint . . . 10
7.4 Computational Cost . . . 10
7.5 Edge Deployment Trade-offs . . . 11

8 Discussion **11**

9 Conclusion **12**

A Layer-Wise Architecture Tables **13**

B Reproducibility and Saved Evidence **13**

Abstract

This report presents a verified DeFungi image-classification study using 9,114 RGB JPEG images from five fungal classes. A deterministic stratified split was used with 6,379 training images, 1,367 validation images, and 1,368 test images. All custom models operated on 64×64 inputs. Model A, a standard convolutional neural network with 101,829 trainable parameters, achieved 0.6959 test accuracy. Model B, a lightweight depthwise-separable CNN with 14,272 trainable parameters, achieved 0.6484 test accuracy while reducing parameters by 85.98% and serialized model size by 82.33% relative to Model A. Two ImageNet transfer-learning models were also evaluated on the same sample membership at 64×64 input: MobileNetV2 achieved 0.6908 test accuracy and EfficientNet-B0 achieved 0.7668. The final comparison shows that EfficientNet-B0 gave the highest measured accuracy, while Model B had the smallest disk footprint, lowest included MAC estimate, and lowest measured CPU batch-1 latency. The results therefore show a practical trade-off: transfer learning improves accuracy, but the custom lightweight model is more attractive when memory and CPU latency are the dominant constraints.

1 Introduction

Edge image-classification systems often need to balance accuracy against memory, computation, and latency. The purpose of this assignment was to design and evaluate convolutional neural networks for a resource-constrained DeFungi classification task, then compare the custom models with lightweight pretrained architectures.

The work was organized around four model families. Model A used standard convolution layers and acted as the stronger custom baseline. Model B used depthwise-separable convolution to reduce the parameter count below the assignment target of 100,000 parameters. MobileNetV2 [2] and EfficientNet-B0 [3] were evaluated as lightweight state-of-the-art transfer-learning alternatives. All results reported here come from saved project artifacts only; no report value is taken from a rerun or from an external group report.

2 Dataset and Data Preparation

2.1 DeFungi Dataset

The dataset used in this project was DeFungi from the UCI Machine Learning Repository [4]. The verified local copy contains 9,114 usable images. All verified images are JPEG, RGB, and 500×500 pixels. No unreadable images were recorded in the dataset verification output.

2.2 Dataset Classes

The classification problem has five target classes: H1, H2, H3, H5, and H6. Table 1 gives the verified class counts and split allocation. The class distribution is imbalanced: H1 has 4,404 images, while H6 has 739 images.

Table 1: Verified DeFungi class counts and stratified split allocation.

Class	Total	Train	Validation	Test
H1	4404	3082	661	661
H2	2334	1634	350	350
H3	819	573	123	123
H5	818	573	122	123
H6	739	517	111	111
Total	9114	6379	1367	1368

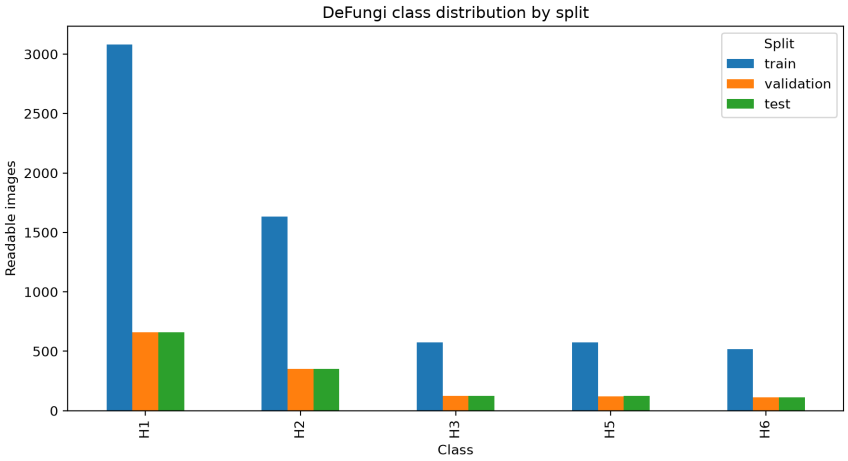


Figure 1: Verified class distribution generated from the saved dataset summary.

2.3 64×64 Input

The assignment required compact image inputs for a resource-constrained setting. The custom data pipeline therefore resized all images to 64×64 with three channels and normalized pixel values to $[0, 1]$. The Session 3 transfer-learning runs also used 64×64 input because both requested ImageNet backbones loaded correctly at this resolution. This preserves consistent sample membership and avoids silently switching to a larger ImageNet input size.

2.4 Stratified 70/15/15 Split

A deterministic stratified split was created once using seed 42. The split proportions were approximately 70% training, 15% validation, and 15% test for each class. The saved verification files report 9,114 unique paths, no duplicate split paths, and the split counts shown in Table 1. The test set was used only for final evaluation.

2.5 Preprocessing and Augmentation

For custom CNN training, images were decoded as RGB, resized to 64×64, converted to floating point, and scaled by 1/255. The saved training evidence does not depend on a new split or on test-time selection. For pretrained models, the same split membership was used, but the Keras preprocessing function associated with each backbone was applied. MobileNetV2 used `mobilenet_v2.preprocess_input`; EfficientNet-B0 used the Keras EfficientNet preprocessing entry point recorded in the metrics file.

3 Custom CNN Architectures

3.1 Model A

Model A is a standard CNN baseline using three 3×3 convolution blocks with ReLU activation, max pooling after each convolution, global average pooling, and a small dense classifier. It has 101,829 trainable parameters. The use of global average pooling avoids a large flattening layer and keeps the classifier compact.

Table 2: Model A architecture summary from the verified architecture CSV.

Layer	Type	Output shape	Activation	Parameters
Input	InputLayer	$64 \times 64 \times 3$	–	0
Conv1	Conv2D 32, 3×3	$64 \times 64 \times 32$	ReLU	896
Pool1	MaxPool 2×2	$32 \times 32 \times 32$	–	0
Conv2	Conv2D 64, 3×3	$32 \times 32 \times 64$	ReLU	18496
Pool2	MaxPool 2×2	$16 \times 16 \times 64$	–	0
Conv3	Conv2D 128, 3×3	$16 \times 16 \times 128$	ReLU	73856
Pool3	MaxPool 2×2	$8 \times 8 \times 128$	–	0
GAP	GlobalAveragePooling2D	128	–	0
Dense	Dense 64	64	ReLU	8256
Output	Dense 5	5	Softmax	325
Total				101829

3.2 Model B

Model B follows the same high-level classifier pattern but replaces standard convolution with depthwise-separable convolution. It has 14,272 trainable parameters, well below the 100,000-parameter target. The convolution stages use 32, 64, and 96 output channels, followed by global average pooling and a 48-unit dense layer.

Table 3: Model B architecture summary from the verified architecture CSV.

Layer	Type	Output shape	Activation	Parameters
Input	InputLayer	$64 \times 64 \times 3$	–	0
SepConv1	SeparableConv2D 32	$64 \times 64 \times 32$	ReLU	155
Pool1	MaxPool 2×2	$32 \times 32 \times 32$	–	0
SepConv2	SeparableConv2D 64	$32 \times 32 \times 64$	ReLU	2400
Pool2	MaxPool 2×2	$16 \times 16 \times 64$	–	0
SepConv3	SeparableConv2D 96	$16 \times 16 \times 96$	ReLU	6816
Pool3	MaxPool 2×2	$8 \times 8 \times 96$	–	0
GAP	GlobalAveragePooling2D	96	–	0
Dense	Dense 48	48	ReLU	4656
Output	Dense 5	5	Softmax	245
Total				14272

3.3 Depthwise Separable Convolution

For a standard convolution with kernel size $K_h \times K_w$, C_{in} input channels, C_{out} output channels, and one bias per output channel, the parameter count used in this project is

$$P_{\text{std}} = (K_h K_w C_{in} + 1) C_{out}. \tag{1}$$

For Keras `SeparableConv2D` with bias enabled, the saved implementation uses one depthwise spatial kernel per input channel, a 1×1 pointwise convolution, and one bias per output channel:

$$P_{\text{sep}} = K_h K_w C_{in} + C_{in} C_{out} + C_{out}. \tag{2}$$

The reduction percentage is

$$R = \frac{P_{\text{std}} - P_{\text{sep}}}{P_{\text{std}}} \times 100\%. \tag{3}$$

Table 4: Verified parameter comparison for standard and depthwise-separable stages.

Stage	Standard params	Depthwise	Pointwise+bias	Reduction
3→32	896	27	128	82.70%
32→64	18496	288	2112	87.02%
64→96	55392	576	6240	87.69%

3.4 Parameter Calculation

The full-model parameter reduction from Model A to Model B is 87,557 parameters:

$$\frac{101829 - 14272}{101829} \times 100 = 85.98\%. \tag{4}$$

This reduction is mostly due to replacing expensive spatial convolution kernels with depthwise and pointwise operations in Model B.

3.5 Hardware-Aware Activation Selection

Both custom models used ReLU in hidden layers. ReLU is appropriate for hardware-aware design because it mainly computes $\max(0, x)$ and does not require exponentials. The output layer used softmax because the task is single-label five-class classification.

4 Optimizer Selection and Tuning

4.1 SGD

The controlled optimizer pilot included stochastic gradient descent with learning rate 0.01. It reached a final validation loss of 1.4039 after five pilot epochs.

4.2 SGD with Momentum

Momentum adds a velocity term that accumulates previous updates:

$$v_t = \mu v_{t-1} - \eta \nabla_\theta J(\theta_t), \tag{5}$$

$$\theta_{t+1} = \theta_t + v_t. \tag{6}$$

The momentum pilot used $\mu = 0.9$ and learning rate 0.01. It reached a final validation loss of 1.3385.

4.3 Selected Adaptive Optimizer

Adam [1] was the selected adaptive optimizer. The pilot used learning rate 0.001 and reached the lowest final validation loss among the tested optimizers.

4.4 Experimental Comparison

The optimizer comparison was a controlled pilot on Model B using 30 training batches per epoch and the full validation split. It was not treated as a final test result. Table 5 summarizes the final pilot epoch for each optimizer.

Table 5: Final epoch of the verified optimizer pilot.

Optimizer	LR	Momentum	Final validation loss	Final validation accuracy
Adam	0.001	0.0	1.2509	0.5091
SGD	0.010	0.0	1.4039	0.4835
SGD + momentum	0.010	0.9	1.3385	0.4835

Compared with plain SGD, momentum lowered the pilot's final validation loss from 1.4039 to 1.3385, while validation accuracy stayed at 0.4835. Accumulating updates can smooth progress in consistent gradient directions, but this short pilot showed no accuracy gain over plain SGD. Adam had both the lowest validation loss and the highest validation accuracy in this comparison. These conclusions are limited to the pilot settings and five-epoch duration.

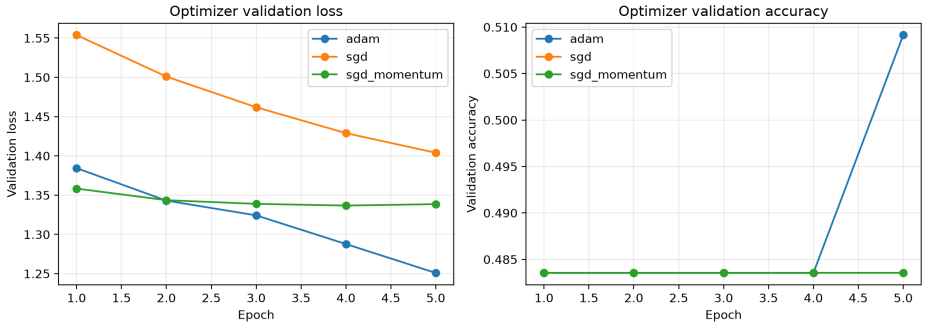


Figure 2: Optimizer pilot curves generated from the saved optimizer comparison artifact.

4.5 Selected Optimizer

Adam was selected because it had the lowest final validation loss in the controlled optimizer comparison. The final Model A and Model B training runs therefore used Adam.

5 Custom CNN Training and Evaluation

5.1 Training Setup

Model A and Model B were trained for 20 epochs with batch size 32 on the fixed DeFungi split. The validation set was used during training monitoring, and the test set was used only for final reported metrics. Accuracy, macro precision, macro recall, and macro F1 were computed from the saved test predictions.

For a test set of N samples, accuracy is

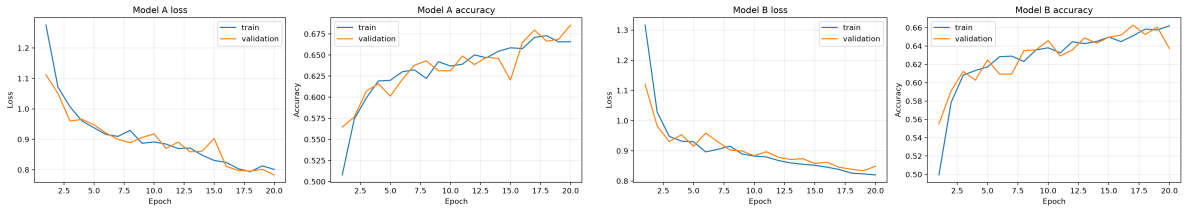
$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}(\hat{y}_i = y_i). \tag{7}$$

Macro precision and macro recall average per-class precision and recall equally across the five classes:

$$\text{Macro Precision} = \frac{1}{C}\sum_{c=1}^{C}\frac{TP_c}{TP_c + FP_c}, \tag{8}$$

$$\text{Macro Recall} = \frac{1}{C}\sum_{c=1}^{C}\frac{TP_c}{TP_c + FN_c}. \tag{9}$$

5.2 Training Curves



(a) Model A **(b)** Model B

Figure 3: Training and validation curves for the custom CNNs.

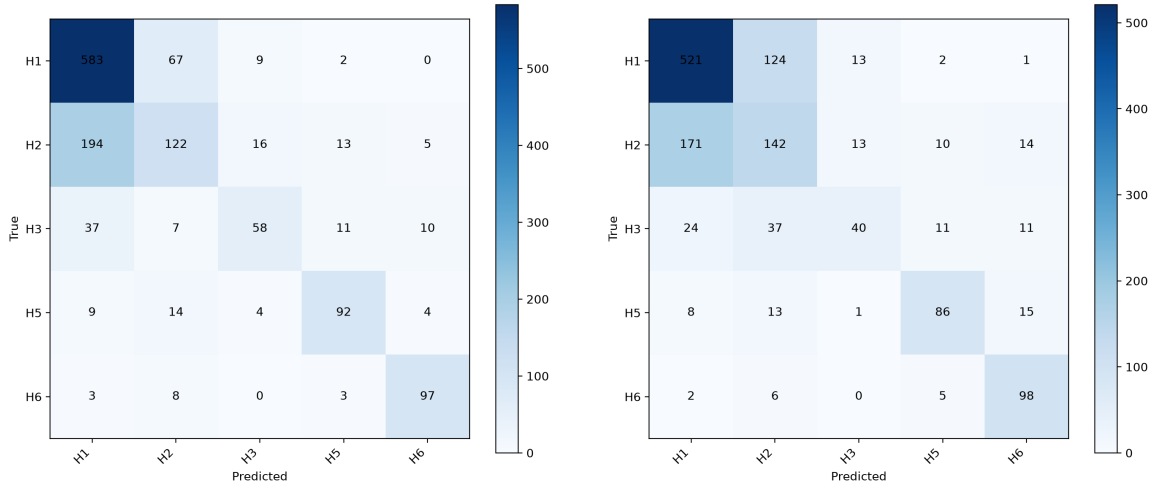
5.3 Test Results

Table 6 reports the verified custom CNN test metrics. Model A achieved higher test accuracy and macro recall than Model B, but Model B used far fewer parameters.

Table 6: Verified custom CNN test metrics.

Model	Accuracy	Macro precision	Macro recall	Macro F1
Model A	0.6959	0.7057	0.6648	0.6750
Model B	0.6484	0.6430	0.6202	0.6209

5.4 Confusion Matrices



(a) Model A **(b)** Model B

Figure 4: Verified custom CNN confusion matrices on the canonical test set.

The most visible confusion pattern is between H1 and H2. Model A classified 583 of 661 H1 test images correctly but assigned 194 H2 images to H1. Model B also favored H1 strongly: it classified 521 H1 images correctly, but 171 H2 images were predicted as H1. H3 was difficult for both custom models, with recalls of 0.4715 for Model A and 0.3252 for Model B. H6 was comparatively strong, with recalls of 0.8739 for Model A and 0.8829 for Model B.

5.5 Model A vs Model B Resource Comparison

The MAC estimates in this report include convolution, depthwise convolution, separable convolution components where applicable, and dense MACs. Bias, activation, pooling, normalization, and data-loading costs are not included. For a convolution layer,

$$\text{MACs}_{\text{conv}} = H_{out}W_{out}C_{out}K_hK_wC_{in}. \tag{10}$$

For dense layers,

$$\text{MACs}_{\text{dense}} = N_{in}N_{out}. \tag{11}$$

Table 7: Verified custom model resource comparison. Serialized size is KiB.

Model	Params	Size	MACs	Mean epoch (s)	CPU mean (ms)
Model A	101829	1241.37	41296192	12.8958	2.8241
Model B	14272	219.41	4621040	7.8730	1.5922

Relative to Model A, Model B reduced parameters by 85.98%, serialized model size by 82.33%, and the included MAC estimate by 88.81%. The measured CPU batch-1 mean latency also decreased by 43.62%, from 2.8241 ms to 1.5922 ms. Runtime did not decrease as much as MACs, which is an important observation for edge deployment: framework overhead, memory access, and available optimized kernels also affect latency.

6 Lightweight State-of-the-Art Models

6.1 MobileNetV2

MobileNetV2 uses inverted residual blocks and depthwise separable convolutions to reduce computation while preserving representational capacity [2]. In this project, the ImageNet pretrained MobileNetV2 feature extractor was adapted to a five-class DeFungi head.

6.2 EfficientNet-B0

EfficientNet-B0 is the baseline EfficientNet model family member, designed around compound scaling of depth, width, and resolution [3]. The project used ImageNet pretrained weights and replaced the classifier with a five-class DeFungi head.

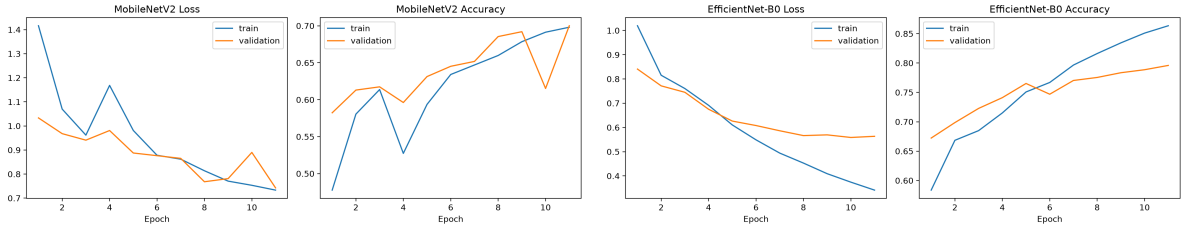
6.3 Transfer-Learning Strategy

Both pretrained models used a two-stage transfer-learning strategy. Stage 1 froze the feature extractor and trained the new classifier head for three epochs. Stage 2 fine-tuned an upper subset of the backbone with a smaller learning rate. Validation loss selected the saved checkpoint, and the test set was not used for model selection.

Table 8: Verified transfer-learning setup.

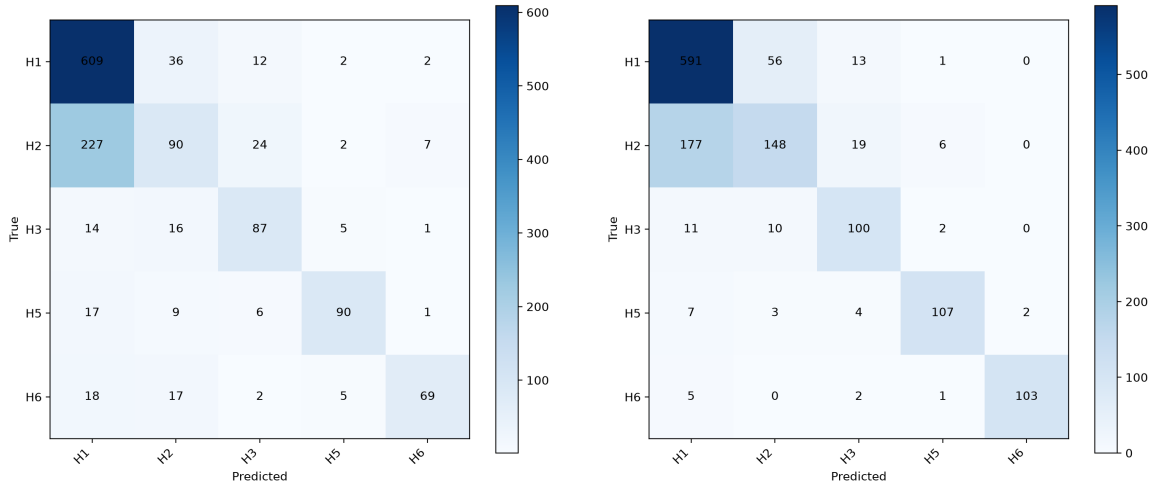
Model	Input	Head epochs	Fine-tune epochs	Head LR	Fine-tune LR
MobileNetV2	64×64	3	8	0.001	0.0001
EfficientNet-B0	64×64	3	8	0.001	0.0001

6.4 Results



(a) MobileNetV2 **(b)** EfficientNet-B0

Figure 5: Training and validation curves for the 64×64 transfer-learning runs.



(a) MobileNetV2 **(b)** EfficientNet-B0

Figure 6: Verified transfer-learning confusion matrices on the canonical test set.

Table 9: Verified transfer-learning test metrics, total parameter counts, and serialized model sizes. Model size is shown in decimal MB (bytes divided by 10^6).

Model	Parameters	Size (MB)	Accuracy	Macro precision	Macro recall	Macro F1
MobileNetV2	2264389	22.53	0.6908	0.7232	0.6478	0.6672
EfficientNet-B0	4055976	33.43	0.7668	0.8099	0.7856	0.7895

EfficientNet-B0 gave the strongest test accuracy and macro recall of all four evaluated models. Its confusion matrix shows stronger recognition of H3, H5, and H6 than the custom models, but H2 remains challenging: 177 H2 test samples were still classified as H1. MobileNetV2 was close to Model A in accuracy but did not improve over Model A under the verified 64×64 transfer-learning setup.

7 Final Comparison

7.1 Accuracy

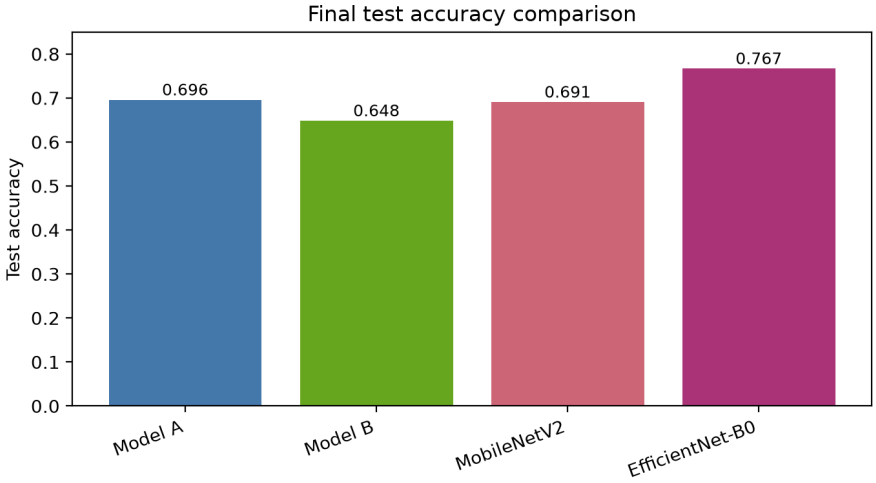


Figure 7: Final test accuracy comparison from the verified final comparison CSV.

EfficientNet-B0 achieved the highest test accuracy at 0.7668. Model A was the best custom model at 0.6959. MobileNetV2 achieved 0.6908, slightly below Model A in this 64×64 run. Model B achieved 0.6484, giving up 0.0475 absolute accuracy relative to Model A in exchange for a much smaller footprint.

7.2 Parameter Count

Table 10 gives the final comparison over all four models. Model B is the smallest by parameter count, with 14,272 parameters. The pretrained models contain millions of parameters because their ImageNet feature extractors are retained.

7.3 Memory Footprint

Model B has the smallest serialized model artifact at 219.41 KiB. Model A is 1241.37 KiB. MobileNetV2 and EfficientNet-B0 are much larger at 22000.13 KiB and 32650.04 KiB respectively. These serialized sizes are measured saved-model artifact sizes, not only theoretical FP32 parameter storage.

7.4 Computational Cost

The final comparison includes the measured CPU batch-1 latency and the included MAC estimate. Model B has the lowest included MAC estimate and the lowest measured CPU latency. MobileNetV2 has fewer included MACs than Model A in this counting method, but its measured CPU latency is far higher on this machine. This demonstrates that MAC count alone does not determine runtime.

Table 10: Final verified comparison. Serialized model size is KiB; latency is CPU batch-1 in milliseconds.

Model	Params	Size	MACs	Mean latency	P95 latency	Accuracy	Macro P	Macro R
Model A	101829	1241.37	41296192	2.8241	3.2042	0.6959	0.7057	0.6648
Model B	14272	219.41	4621040	1.5922	2.2656	0.6484	0.6430	0.6202
MobileNetV2	2264389	22000.13	24454912	170.1065	198.2038	0.6908	0.7232	0.6478
EfficientNet-B0	4055976	32650.04	31972992	322.2855	357.6918	0.7668	0.8099	0.7856

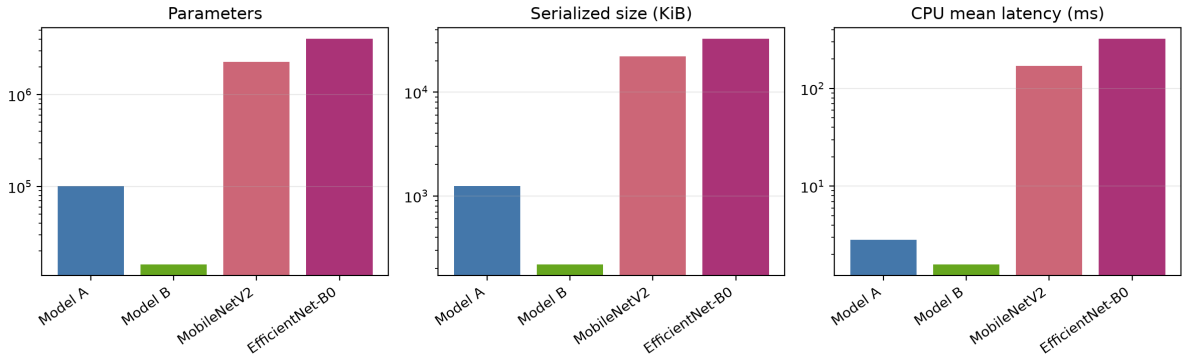


Figure 8: Final resource comparison. The y-axes are logarithmic to show custom and pretrained models in one figure.

7.5 Edge Deployment Trade-offs

There is no single universally best model in the verified results. EfficientNet-B0 is the accuracy leader, but it has the largest parameter count, largest serialized model, and slowest measured CPU latency. Model B is the resource leader, but its accuracy is lower than Model A and EfficientNet-B0. For a very tight memory or latency budget, Model B is the strongest candidate among the measured models. For a less constrained device where accuracy matters more than disk footprint and CPU latency, EfficientNet-B0 is more attractive.

8 Discussion

The custom-model comparison shows the intended effect of depthwise-separable convolution clearly. Model B reduced parameters by 85.98%, serialized size by 82.33%, and the included MAC estimate by 88.81% compared with Model A. Its CPU mean latency also decreased, but only by 43.62%. Therefore, the empirical runtime result follows the same direction as the MAC reduction, but not the same scale.

The accuracy cost of the lightweight custom design was measurable. Model A reached 0.6959 accuracy, while Model B reached 0.6484. This difference is consistent with Model B having fewer channels and fewer trainable parameters. The confusion matrices show that H1 was generally easier because it is also the largest class. H2 was often confused with H1 by all models. H3 was especially weak for the custom models, while EfficientNet-B0 improved H3 recall to 0.8130.

The pretrained models show the benefit and cost of transferred ImageNet representations. EfficientNet-B0 improved the best custom accuracy by 0.0709 absolute accuracy. However, the serialized EfficientNet-B0 model was 32650.04 KiB, compared with 219.41 KiB for Model B. MobileNetV2 did not outperform Model A in this run, but it still had a much larger serialized footprint and much higher measured CPU latency than both custom CNNs.

Parameter count alone should not be used as a complete edge-performance claim. It does not directly measure RAM usage, cache behavior, kernel launch overhead, implementation quality, or energy. MAC estimates also exclude operations such as activation functions, pooling, normalization, and memory movement. The CPU latency measurements are useful for comparing these saved Keras models on this computer, but they are not microcontroller measurements.

The main limitations are the use of a single split, a single seed, limited hyperparameter search, possible class imbalance, and no physical edge-device energy measurement. The transfer-learning results also rely on external ImageNet prior knowledge. Finally, 64×64 input is much smaller than standard ImageNet transfer-learning practice, so the pretrained models were intentionally evaluated under an assignment-consistent but resolution-constrained interpretation.

9 Conclusion

The assignment objective was achieved using verified DeFungi artifacts. Model A provided the strongest custom CNN accuracy, while Model B satisfied the lightweight design goal with 14,272 parameters and the smallest measured resource footprint. EfficientNet-B0 achieved the best overall accuracy, but with the largest serialized model and highest CPU latency among the four evaluated models. The final recommendation depends on deployment priorities: Model B is the most appropriate measured option for tight memory and latency constraints, while EfficientNet-B0 is preferable when accuracy is the dominant requirement and the platform can support the extra resource cost.

References

[1] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015.

[2] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.

[3] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.

[4] UCI Machine Learning Repository. DeFungi Dataset. `https://archive.ics.uci.edu/dataset/773/defungi`, n.d. Dataset used for EN3150 Assignment 03.

A Layer-Wise Architecture Tables

Table 11: Complete Model A layer table from the verified architecture CSV.

Layer	Class	Output shape	Kernel	Stride	Activation	Params
input_layer	InputLayer	None x 64 x 64 x 3	–	–	–	0
conv2d	Conv2D	None x 64 x 64 x 32	3x3	1x1	ReLU	896
max_pooling2d	MaxPooling2D	None x 32 x 32 x 32	–	2x2	–	0
conv2d_1	Conv2D	None x 32 x 32 x 64	3x3	1x1	ReLU	18496
max_pooling2d_1	MaxPooling2D	None x 16 x 16 x 64	–	2x2	–	0
conv2d_2	Conv2D	None x 16 x 16 x 128	3x3	1x1	ReLU	73856
max_pooling2d_2	MaxPooling2D	None x 8 x 8 x 128	–	2x2	–	0
global_average_pooling2d	GlobalAveragePooling2D	None x 128	–	–	–	0
dense	Dense	None x 64	–	–	ReLU	8256
dense_1	Dense	None x 5	–	–	Softmax	325

Table 12: Complete Model B layer table from the verified architecture CSV.

Layer	Class	Output shape	Kernel	Stride	Activation	Params
input_layer_1	InputLayer	None x 64 x 64 x 3	–	–	–	0
separable_conv2d	SeparableConv2D	None x 64 x 64 x 32	3x3	1x1	ReLU	155
max_pooling2d_3	MaxPooling2D	None x 32 x 32 x 32	–	2x2	–	0
separable_conv2d_1	SeparableConv2D	None x 32 x 32 x 64	3x3	1x1	ReLU	2400
max_pooling2d_4	MaxPooling2D	None x 16 x 16 x 64	–	2x2	–	0
separable_conv2d_2	SeparableConv2D	None x 16 x 16 x 96	3x3	1x1	ReLU	6816
max_pooling2d_5	MaxPooling2D	None x 8 x 8 x 96	–	2x2	–	0
global_average_pooling2d_1	GlobalAveragePooling2D	None x 96	–	–	–	0
dense_2	Dense	None x 48	–	–	ReLU	4656
dense_3	Dense	None x 5	–	–	Softmax	245

B Reproducibility and Saved Evidence

Table 13: Mapping from report requirements to saved project evidence.

Requirement	Saved evidence path
Dataset and split	`data/processed/splits.csv`, `results/verification/split_verification.json`
Dataset summary	`results/tables/dataset_summary.csv`, `results/metrics/dataset_summary.json`
Model A architecture	`results/tables/model_a_architecture.csv`, `results/metrics/model_a_summary.txt`
Model B architecture	`results/tables/model_b_architecture.csv`, `results/metrics/model_b_summary.txt`
Parameter checks	`results/verification/model_parameter_checks.json`
Optimizer comparison	`results/tables/optimizer_comparison.csv`, `results/figures/optimizer_comparison.png`
Custom training histories	`results/raw/model_a/history.csv`, `results/raw/model_b/history.csv`
Custom test predictions	`results/raw/model_a/test_predictions.csv`, `results/raw/model_b/test_predictions.csv`
Custom confusion matrices	`results/raw/model_a/confusion_matrix.csv`, `results/raw/model_b/confusion_matrix.csv`
Pretrained results	`results/raw/mobilenetv2/metrics.json`, `results/raw/efficientnetb0/metrics.json`
Pretrained histories	`results/raw/mobilenetv2/history.csv`, `results/raw/efficientnetb0/history.csv`
Resource comparison	`results/tables/final_comparison.csv`
MAC evidence	`results/tables/custom_model_macs.csv`, `results/tables/pretrained_transfer_macs.csv`
CPU latency evidence	`results/tables/custom_latency.csv`, `results/tables/pretrained_transfer_latency.csv`
Environment and pipeline checks	`results/metrics/environment_info.json`, `results/verification/custom_pipeline_check.json`